KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

DIGITAL WALLET AND BALANCE MANAGEMENT SYSTEM

Submitted by

NAME:DHANALAKSHMI S

REGISTER NO:25BIT013

Under the Guidance of

Trainer Name

DHILEEBAN A

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled “Digital Wallet and Balance Management System**”** submitted in partial fulfillment of the requirements for the course [DHANALAKSHMI/B.SC.IT**]** is a **record of my original work** carried out under the guidance of **[DHILEEBAN A]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

DHANALAKSHMI S

(Your Name)

CERTIFICATE

This is to certify that the project entitled **"Digital Wallet and Balance Management System"** is a **bona fide work** carried out by **[DHANALAKSHMI] [B.SC.IT-A/]**, [**Department Name]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

_____ _____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Digital Wallet and Balance Management System** is a Java-based application designed to automate the management of digital wallets, enabling users to store funds, make payments, transfer money, and track transaction histories electronically. In many financial interactions, users rely on cash or card-based systems that lack real-time tracking and unified management. This project aims to simplify financial operations by providing an automated system that manages wallet balances, processes transactions, and generates accurate statements.

The system allows users to create digital wallets, deposit or withdraw funds, transfer money between wallets, and automatically track all transactions with timestamps. The application is developed using **Core Java programming concepts**, including classes, objects, methods, conditional statements, loops, and exception handling. The project can be implemented either as a **console-based application or using Java Swing for a graphical user interface**.

The main objective of the system is to improve financial transaction efficiency, reduce human errors, and provide instant access to wallet balances. The system generates a clear and structured statement that displays all transactions, credits, debits, and the current available balance.

This project demonstrates the practical implementation of **object-oriented programming concepts in Java** and provides a basic foundation for developing more advanced restaurant management systems in the future.

Introduction

With the advancement of computer technology, automated digital wallet systems have become essential for improving efficiency and accuracy in financial management. A computerized wallet system helps users quickly process payments, transfer funds automatically, and maintain organized records of all financial transactions.

The **Digital Wallet and Balance Management System** is a Java-based application designed to automate the process of wallet management and transaction processing. The system allows the user to create a digital wallet, deposit or withdraw funds, transfer money to other wallets, and automatically update the balance after each transaction. The application generates a clear and structured statement displaying transaction type, amount, timestamp, and current balance.

This project is developed using **Core Java programming concepts** such as classes, objects, methods, conditional statements, loops, and exception handling. The application can be implemented either as a **console-based program or using Java Swing for a graphical user interface (GUI)**.

The primary goal of this system is to reduce manual effort, improve transaction accuracy, and provide instant financial management to users. This project also demonstrates how Java programming can be applied to develop simple real-world financial applications.

1.1 Objective

The main objective of the **Digital Wallet and Balance Management System** is to develop a simple and efficient wallet management application using Core Java that automates the process of fund management and transaction processing for digital payments.

The specific objectives of this project are:

- To design and develop a **Java-based digital wallet management system**.
- To automate the **processing of deposits, withdrawals, and balance updates**.
- To reduce **manual errors in financial transaction processing**.
- To provide a **user-friendly system for restaurant staff** to generate invoices quickly.
- To demonstrate the implementation of **Core Java programming concepts** such as classes, objects, methods, loops, and conditional statements.
- To generate a **clear and organized transaction statement for users**.

1.2 Overview

The **Digital Wallet and Balance Management System** is a menu-driven application developed using Java programming language. The system provides a digital platform where users can manage their wallets, process transactions, and view balances automatically.

The system displays wallet information along with the current balance. The user can choose to deposit funds, withdraw money, transfer to another wallet, or view the transaction history. The program updates the wallet balance after each operation and generates a transaction statement.

The system helps simplify financial management by eliminating manual record-keeping and reducing the chances of transaction errors. The application can be developed as a **console-based program or a graphical user interface using Java Swing**.

This project demonstrates the practical use of Java programming in developing real-world applications for digital financial environments such as e-commerce and mobile payment systems.

1.3 Features

The **Digital Wallet and Balance Management System** includes the following main features:

• **Wallet Creation** – Displays a list of food items available in the restaurant along with their prices.

• **Deposit and Withdrawal** – Allows the user to select food items from the menu and enter the quantity ordered by the customer.

• **Fund Transfer** – Automatically calculates the total amount based on the selected items and quantities.

• **Balance Inquiry** – Calculates applicable taxes such as GST and adds them to the total bill.

• **Transaction History** – Generates a detailed bill showing item names, quantities, prices, subtotal, tax, and final amount.

• **User-Friendly Interface** – Provides a simple and easy-to-use menu-driven interface.

• **Secure Transaction Processing** – Reduces the time required to generate customer bills.

2.PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are identified. This stage helps in understanding how the system should work and what features are required to solve the problem efficiently.

The analysis phase focuses on identifying the current issues in the existing system and proposing a computerized solution that improves efficiency and accuracy.

2.1 Problem Definition

In many financial transactions, users manage their funds manually using cash or physical records. This traditional method creates several challenges.

Some common problems include:

• Errors in bill calculation
• Time-consuming billing process
• Difficulty in managing multiple orders
• Lack of proper billing records
• Inefficient service during busy hours

Manual financial systems also increase the workload for users and can lead to dissatisfaction due to inaccurate balance tracking or lost transaction records.

Therefore, there is a need for an automated wallet system that can quickly process transactions, manage multiple wallets, and generate accurate financial statements.

The **Digital Wallet and Balance Management System** is designed to solve these problems by providing a computerized solution that automates the wallet management process.

2.2 Project Overview

The **Digital Wallet and Balance Management System** is a Java-based application developed to simplify the management of digital wallets and financial transactions.

The system provides a menu-driven interface where users can create wallets, deposit or withdraw funds, transfer money, and view transaction history. The system automatically updates the wallet balance after each operation and generates a final statement.

The system includes the following functionalities:

• Displaying the restaurant menu with item prices
• Selecting food items and entering quantity
• Calculating item cost automatically
• Computing subtotal and tax amount
• Generating the final bill for the customer

This project is implemented using **Core Java programming concepts** and can be developed as either a **console-based program or a Java Swing GUI application**.

2.3 Scopes and Limitations

Scope

The scope of the **Digital Wallet and Balance Management System** includes the following:

- Managing restaurant menu items
- Processing customer food orders
- Calculating the total bill automatically
- Generating a detailed invoice for the customer
- Demonstrating the practical use of Java programming concepts

The system can be used in:

- Restaurants
- Hotels
- Cafeterias
- Food courts
- Small food businesses

3.SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The **Digital Wallet and Balance Management System** is designed as a menu-driven application where users can manage wallets, process transactions, and view statements automatically.

The system is divided into several modules to perform specific tasks. Each module is responsible for handling a particular function of the system.

3.1 Modules

The main modules of the **Digital Wallet and Balance Management System** are explained below:

1. Menu Display Module

This module is responsible for creating and managing digital wallets. It allows users to initialize a wallet with a unique ID and starting balance.

Example Wallet Creation:

1. Veg Fried Rice ₹120
2. Chicken Biryani ₹200
3. Paneer Butter Masala ₹180
4. Butter Naan ₹40
5. Cold Drink ₹30

The wallet details are displayed when the program starts so that the user can easily manage their account.

2. Order Selection Module

This module allows the user to add funds to their wallet or withdraw money from the available balance.

Functions of this module include:

• Accepting user input for item selection
• Accepting quantity of selected items
• Storing order details temporarily

The system validates each transaction to ensure the wallet has sufficient funds before processing a withdrawal.

3. Bill Calculation Module

This module handles money transfers between two wallets.

The system performs:

• Total price of each item
• Subtotal of all selected items
• Applicable tax (GST)
• Final payable amount

The transfer is performed atomically using Java programming logic to ensure both wallets are updated correctly.

Example Transfer:

Item Total = Price × Quantity
GST = 5% of Subtotal
Final Bill = Subtotal + GST

4. Invoice Generation Module

This module generates a detailed transaction statement for the user.

The statement includes:

• Item name
• Quantity
• Price
• Subtotal
• GST amount
• Final bill amount

Example Statement Output:

------ DIGITAL WALLET ------

Item	Qty	Price
Chicken Biryani	2	400
Cold Drink	2	60

Subtotal	460
GST (5%)	23
Total	483

Thank You Visit Again!

5. Exit Module

This module allows the user to exit the application safely after completing all wallet transactions.

When the user selects the exit option, the system saves all transaction data and terminates the program.

4.METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Digital Wallet and Balance Management System**, a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the needs of restaurant operations.

The main problems identified in manual financial management systems are:

- Manual calculations may lead to billing errors.
- Preparing bills manually takes more time.
- Difficult to handle multiple customer orders efficiently.
- Lack of automated records for transactions.

Based on these problems, the system requirements were defined to develop an automated wallet application that simplifies digital financial operations.

4.2. System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:

• Menu Display Module
• Order Selection Module
• Bill Calculation Module
• Invoice Generation Module
• Exit Module

The program flow and structure are designed so that users can easily interact with the system.

4.3. Program Development

After designing the system, the program is implemented using the **Java programming language**.

The following steps were followed during development:

• Creation of classes and methods
• Implementation of menu-driven logic
• Writing code for bill calculation
• Handling user input
• Displaying the final bill

Core Java concepts such as **classes, objects, loops, conditional statements, and methods** were used during development.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases were used to check the functionality of the system.

Example Test Cases:

Test Case	**Expected Result**
Deposit funds	Balance updated

Test Case	Expected Result
into wallet	correctly
Withdraw more than balance	Insufficient funds error shown
Transfer between wallets	Both wallet balances updated correctly
Invalid input	Error handled properly

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

After successful testing, the results of the system are documented. The system generates the transaction statement based on all wallet operations performed.

The results are demonstrated through program execution and screenshots showing wallet creation, transactions, and balance updates.

Proper documentation is prepared to explain the design, implementation, and working of the system.

4.6. Algorithm

Algorithm for the **Digital Wallet and Balance Management System**:

1. Start the program.
2. Create a digital wallet with owner name and initial balance.
3. Display the wallet menu (Deposit / Withdraw / Transfer / View History / Exit).
4. Accept user's choice and transaction amount.
5. Validate the transaction (check sufficient balance for withdrawals/transfers).
6. Update the wallet balance accordingly.

7. Repeat steps 3–6 for additional items if required.
8. Repeat steps 3–7 until the user selects Exit.
9. Display the final wallet balance and transaction summary.
10.
11.

4.7. Flowchart

Students should draw a flowchart showing program flow.

Example flow:

Start
↓
Create Wallet
↓
Select Operation
↓
Enter Amount
↓
Process Transaction
↓
Update Balance
↓
End

Students can draw using:

- draw.io
- Lucidchart
- MS Visio

5.SOURCE CODE

```
 package wallet;
public class WalletService {
   public void addMoney(Wallet wallet, double amount) {
       wallet.setBalance(wallet.getBalance() + amount);
       wallet.getTransactions().add("Added: " + amount);
       System.out.println("Money added successfully!");
   }
   public void sendMoney(Wallet sender, Wallet receiver, double amount) {
       if (sender.getBalance() >= amount) {
           sender.setBalance(sender.getBalance() - amount);
           receiver.setBalance(receiver.getBalance() + amount);
           sender.getTransactions().add("Sent: " + amount + " to " + receiver.getUserName());
           receiver.getTransactions().add("Received: " + amount + " from " +
 sender.getUserName());
           System.out.println("Transaction successful!");
       } else {
           System.out.println("Insufficient balance!");
       }
   }
   public void checkBalance(Wallet wallet) {
       System.out.println("Balance: " + wallet.getBalance());
   }
   public void showTransactions(Wallet wallet) {
       System.out.println("Transaction History:");
       for (String t : wallet.getTransactions()) {
           System.out.println(t);
       }
```

```
    }
}
package wallet;
import java.util.ArrayList;
import java.util.List;
public class Wallet {
    private String userName;
    private double balance;
    private List<String> transactions;
    public Wallet(String userName) {
        this.userName = userName;
        this.balance = 0;
        this.transactions = new ArrayList<>();
    }
    public String getUserName() {
        return userName;
    }
    public double getBalance() {
        return balance;
    }
    public void setBalance(double balance) {
        this.balance = balance;
    }
    public List<String> getTransactions() {
        return transactions;
    }
}
```

```
package wallet;
import java.util.Scanner;
public class MainApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        WalletService service = new WalletService();
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        Wallet user = new Wallet(name);
        Wallet friend = new Wallet("Friend");
        int choice;
        do {
            System.out.println("\n--- Digital Wallet ---");
            System.out.println("1. Add Money");
            System.out.println("2. Send Money");
            System.out.println("3. Check Balance");
            System.out.println("4. Transaction History");
            System.out.println("5. Exit");
            System.out.print("Choose option: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter amount: ");
                    double add = sc.nextDouble();
                    service.addMoney(user, add);
                    break;
                case 2:
```

```
                System.out.print("Enter amount: ");
                double send = sc.nextDouble();
                service.sendMoney(user, friend, send);
                break;
            case 3:
                service.checkBalance(user);
                break;
            case 4:
                service.showTransactions(user);
                break;
            case 5:
                System.out.println("Thank you!");
                break;
            default:
                System.out.println("Invalid choice!");
        }
    } while (choice != 5);
    sc.close();
  }
}
```

6.INPUT AND OUTPUT SCREENS

INPUT

```
Enter your name: Dhana
Choose option: 1
Enter amount: 1000

Choose option: 3

Choose option: 2
Enter amount: 300

Choose option: 4

Choose option: 5
```

DETAILS

```
Enter your name: Dhana
Choose option: 1
Enter amount: 1000

Choose option: 3

Choose option: 2
Enter amount: 300

Choose option: 4

Choose option: 5
```

OUTPUT

```
Enter your name: Dhana
---- Digital Wallet ---
1. Add Money
2. Send Money
3. Check Balance
4. Transaction History
5. Exit
Choose option: 1
Enter amount: 1000
Money added successfully!
---- Digital Wallet ---
1. Choose option: 3
Balance: 1000.0
Choose option: 2
----- Digital Wallet ---
1. Choose option: 2
Enter amount: 300
Transaction successful!
---- Digital Wallet ---
1. Choose option: 4
Transaction History:
Added: 1000.0
Sent: 300.0 to Friend
---- Digital Wallet ---
1. Choose option: 5
Thank you!
```

```
Enter your name: Dhana
---- Digital Wallet ---
1. Add Money
2. Send Money
3. Check Balance
4. Transaction History
5. Exit
Choose option: 2
Enter amount: 500
Insufficient balance!
---- Digital Wallet ---
1. Choose option: 5
Thank you!
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

The **Digital Wallet and Balance Management System** was developed to automate the management of digital wallets and financial transactions. The system allows users to create wallets, deposit or withdraw funds, transfer money, and view transaction history automatically.

During the development process, Core Java programming concepts such as loops, conditional statements, methods, and object-oriented principles were applied. The system successfully performs bill calculations and generates accurate invoices.

The system was tested with different inputs to ensure that it performs transaction validations correctly. The results show that the system reduces manual effort and improves financial transaction efficiency.

7.2 Conclusion

The **Digital Wallet and Balance Management System** successfully demonstrates the practical application of Core Java programming concepts in developing a real-world financial management application.

The system automates wallet transactions, reduces calculation errors, and provides instant financial management to users. It simplifies digital payment operations by generating accurate transaction statements based on user activities.

This project helps students understand how Java programming can be used to develop practical financial software systems. The project can be further enhanced by adding features such as database connectivity, graphical user interfaces, and real-time payment gateway integration.

8.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com